FRESH Solutions: an adapter and exact-replay harness for metaheuristic benchmarking

Gregory Paradis
UBC FRESH Lab, University of British Columbia, Vancouver, BC, Canada
gregory.paradis@ubc.ca

Abstract

Applied metaheuristic research is slowed less by the choice of search method than by the engineering around it: problem-specific state, exact objective accounting, seed and budget handling, stopping rules, and evidence that another person can replay. FRESH Solutions is a Python harness that makes those concerns part of a small problem-adapter contract. An adapter supplies exact evaluation, feasibility, proposals, and reversible moves; the engine owns resident assignment state, a replay gate, convergence records, and a budget protocol. Two reference adapters are included: Taillard-style tabu search for the quadratic assignment problem (QAP) and iterated Lin–Kernighan for the travelling salesman problem (TSP). We report an experience-oriented validation, not a solver competition. A 40-run QAP framework-use check matched the standalone and production objectives exactly and replayed every incumbent, at 1.24 times the CPU time of a private-array path (95% CI 1.17–1.31). On a 20-instance QAPLIB panel with ten seeds, the tabu reference averaged a 0.301% gap, while a separately implemented Breakout Local Search (BLS) baseline averaged 0.096%; the cluster-by-instance 95% interval for the paired difference was -0.300 to -0.111 percentage points, with BLS lower on 15 of 16 non-tied instance means. A frozen, mixed-depth 20-instance TSPLIB breadth panel reached 94 of 100 known optima; 95 runs stopped on patience and five `brazil58` runs hit a wall-clock cap at the optimum. We also report a negative result for a generic adapter-only search layer. The contribution is the harness, the replay contract, and the reference adapters, not a state-of-the-art claim. The artifact is public as release candidate `p14-evidence-rc11`; a DOI is pending.

Keywords: reproducible benchmarking; metaheuristics; quadratic assignment; travelling salesman; research software

1 Introduction

Applying a known metaheuristic to a new combinatorial problem is often limited by the work around the search rather than by the search itself. A team must define a move representation, maintain incremental state, preserve feasibility, compute the objective exactly, choose a stopping rule, keep seeds and budgets, and produce a record that a reviewer can inspect. These concerns are rebuilt for each project, and they are easy to get subtly wrong. The resulting comparisons are difficult to reproduce even when the algorithm is standard.

The benchmarking literature has treated parts of this problem. Hooker argued that algorithm comparisons should test hypotheses about why a method works [1]; Barr et al. proposed a reporting discipline for heuristic experiments [2]; McGeoch described an experimental method for algorithm simulation [3]; and Beiranvand et al. collect practices for comparing optimisation algorithms, including budget accounting, multiple seeds, and distributional reporting [4, 5]. COCO standardises the comparison of continuous optimisers [6], and irace standardises algorithm configuration [7]. The reproducible-research literature makes the same point: a result depends on the code, environment, and workflow, not only on a seed [8–12], and containers reduce the software part of that dependence [13, 14].

Public benchmark libraries provide instances and reference values: QAPLIB for the quadratic assignment problem [15] and TSPLIB for the travelling salesman problem [16]. General metaheuristic frameworks expose a solution representation, an objective, and a neighbourhood, then supply search operators [17, 18]. What

they typically do not own is the evidence boundary: the rule that a stored incumbent is re-evaluated by the problem adapter before it is reported, and that the stop reason and denominator travel with the result. That boundary is the subject of this paper.

FRESH Solutions is a response to that engineering problem. It provides a problem-adapter contract, exact resident state and reversible moves, local and population search paths, exact replay, convergence records, and reference implementations for QAP and TSP. The paper asks a narrow question: can a user add a problem-specific neighbourhood or reference implementation and obtain credible, replayable evidence without rebuilding the surrounding machinery? We answer with a bounded framework-use experiment, two reference panels, a broad screen, and a negative result. We do not claim that the generic search layers are competitive, that the QAP reference is state of the art, or that the harness has been tested by a third-party adapter author.

The paper makes four contributions. First, it specifies a small adapter contract that separates a problem's exact state, moves, and feasibility from the search layer. Second, it specifies a replay and evidence protocol: an incumbent is not a result unless a new adapter evaluation reproduces the stored objective, and the record names the instance, seed, budget, stopping reason, and provenance. Third, it provides two reference implementations and reports their full distributions, including tails and cap hits. Fourth, it reports a convergence-controlled framework-use check for QAP state management and a TSP breadth panel, together with the failure modes that the protocol exposes. The contribution is software and evidence design; the empirical panels are validation, not a ranking.

2 Software description

2.1 Design requirements

The harness is built around four goals. *Exactness* means that every reported assignment is a complete discrete assignment and every objective is computed by the adapter, not by a search heuristic's internal counter. *Reversibility* means that a move can be applied and undone without rebuilding the whole assignment, so a local search can test neighbours without allocating a new state for every candidate. *Replayability* means that a result is stored with enough metadata for a third party to re-evaluate the incumbent and recover the same objective and feasibility. *Separation* means that the search layer sees only the adapter interface, so a domain-specific neighbourhood can be added without changing the engine.

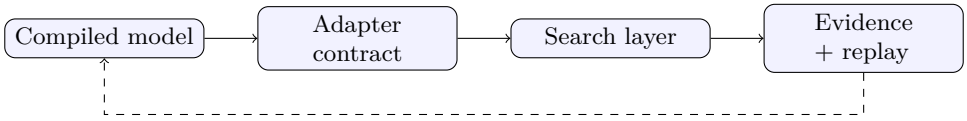


Figure 1: The evidence boundary. The search layer sees only the adapter; every incumbent crosses the replay gate before it becomes a result.

An adapter supplies discrete variables, their options, a complete initial and random assignment, exact and batched evaluation, feasibility, mutation and crossover, and reversible moves. An assignment is a mapping from every variable to one option; for a permutation problem, the adapter owns the permutation invariant. A change records the variable, the option before, the option after, and an optional move kind and segment size. The adapter's `apply` and `undo` operations must be exact inverses on the assignment.

The contract is intentionally smaller than a general modelling language. It does not expose linear constraints, gradients, or symbolic expressions. That restriction keeps it implementable for compiled spatial models and public combinatorial libraries, where the objective is often a domain-specific calculation rather than a closed-form expression. A model that cannot supply exact evaluation and reversible moves is outside the contract. The adapter also supplies `evaluate_batch`, which is used by the population layer and by external baseline adapters; it is a contract for exact evaluations, not a performance promise.

2.2 Resident state and exact replay

The engine exposes `ResidentAssignment`, which holds one complete assignment and routes every accepted move through the adapter's reversible operations, followed by an exact evaluation. The reference modules can also keep a private array and update the objective from an incremental delta; that path is faster and is used for the production QAP tabu scan. The framework-use experiment compares the two representations on the same trajectory, so the measured difference is the cost of state management, not a difference in search strategy.

Replay is more than a checksum. Every reported incumbent is a complete assignment, and the stored assignment must re-evaluate to the stored objective. The record names the public instance and its known optimum or best-known value, the pinned solver version or commit, the seed, the iteration or kick budget, the stopping reason, the exact-replay result, and the wall time; CPU time is recorded where available. A safety-cap hit is recorded as a cap hit; it is not relabelled as convergence. The replay gate constructs a new adapter, evaluates the stored assignment, and compares the objective and feasibility. It does not call the search code, so a record cannot pass by reusing the optimizer's cached objective. For a permutation problem the gate also validates the mapping, and a malformed assignment raises an error.

The evidence format is JSON: per-run records keep the assignment and search counters, and a summary keeps the distribution. The repository also keeps the runner, replay script, instance files or their provenance, and environment. A reader can rerun a campaign, replay the stored incumbents, or compare the distribution without reading the paper prose. A manifest stores SHA-256 hashes for the candidate evidence set; a tag is not created until the maintainer approves the release.

2.3 Stopping and budget protocol

The harness distinguishes four outcomes: plateau stop, safety-cap hit, budget exhaustion, and invalid replay. A plateau stop is a run that met its declared patience rule. A safety-cap hit is a run that reached a wall-clock cap before its patience rule; it is not convergence evidence. Budget exhaustion is a run that reached its iteration or kick cap; it is retained but not called converged. Invalid replay is a record that cannot reproduce its objective; it is removed from the evidence and reported as a defect, not averaged into a table. The denominator is therefore visible, and a mean is always accompanied by the stop counts that produced it.

This taxonomy matters because the budget changes the reported gap. On the four controlled QAP instances, a fixed 10,000-iteration budget gave mean gaps of 0.000%, 0.403%, 0.786%, and 0.792%; the 100,000-iteration patience rule gave 0.000%, 0.030%, 0.077%, and 0.245%. The difference is a longer plateau, not a new algorithm. The TSP campaign shows the same effect: a 20-kick pilot reached the optimum on three of four instances, a 2,000-kick/200-patience run reached 19 of 20 optima, and the 10,000-kick/1,000-patience run reached all 20. A short-budget table would have looked successful and concealed the method's ceiling.

2.4 Reference implementations

The reference implementations are adapter-specific modules. The QAP module uses Taillard-style tabu search [19] over facility swaps, with exact swap deltas, a tabu tenure sampled between $n/2$ and $3n/2$, an aspiration rule for a new incumbent, and Taillard's delta-matrix update for symmetric zero-diagonal instances. Asymmetric instances use an exact general scan. The TSP module uses a bounded candidate-list Lin–Kernighan exchange [20] with 16 nearest neighbours, sequential exchange to depth four, and deterministic double-bridge kicks. The engine still owns the assignment and the replay boundary, but the neighbourhood and acceptance logic are problem-specific. The generic local and population layers [21–23] are retained for smoke tests and contract conformance; they are not the source of the headline results.

3 Validation and quality control

3.1 Framework use and overhead

The first check asks whether a QAP tabu path held in resident state preserves the search trajectory. The framework path repeats the reference scan, tabu memory, start, seed, delta matrix, tenure, patience, and

iteration cap, but applies every accepted swap through the adapter and calls the exact evaluator. Both paths run on four QAPLIB instances, ten seeds each, with a 100,000-iteration patience rule and a 1,000,000-iteration cap. All 40 runs stopped on patience, all 40 framework objectives matched the standalone and production objectives, and all 40 incumbents replayed exactly.

The measured CPU overhead is 1.24 (95% bootstrap CI 1.17–1.31); per-instance means range from 1.17 to 1.33 . This is a real cost, and it has a clear source: the standalone path mutates an array and updates the objective from the delta, while the framework path routes the swap through the adapter and calls the exact evaluator. The overhead is measured against the pure-Python private-array path, not against the Numba-accelerated production path; production CPU time was not recorded, so 1.24 is not an overhead against the fastest path. Even with that caveat, the result is the narrow framework claim: resident state does not change the trajectory, at a measured state-management cost.

3.2 QAP convergence panel and independent baseline

The QAP quality panel uses 20 QAPLIB instances, ten seeds per instance, the same 100,000-iteration patience and 1,000,000-iteration cap. All 200 tabu runs stopped on patience, all 200 replayed exactly, and 73 reached the known optimum. The mean gap was 0.301%, the median 0.043%, and the worst 5.695%. A separately implemented Breakout Local Search baseline [24] used the same instances, seeds, start, declared move budget, and patience. All 200 BLS runs stopped on patience and replayed exactly; BLS reached 161 optima, with a mean gap of 0.096% and a worst gap of 1.369%. Table 1 summarises both arms by instance family.

Table 1: QAPLIB panel: tabu reference versus separately implemented Breakout Local Search (BLS). Each arm uses 20 instances, ten seeds, a 100,000-iteration or 100,000-move patience rule, and a 1,000,000-unit cap. All 400 runs plateau-stopped and replayed exactly. Gaps are percentages to the known optimum; optima are runs at the known optimum.

Family	Instances	Tabu mean	Tabu optima	BLS mean	BLS optima
`tai` (type II)	7	0.421%	32/70	0.220%	40/70
`bur` (type I)	8	0.234%	0/80	0.000%	80/80
`chr` (type III)	5	0.240%	41/50	0.077%	41/50
All	20	0.301%	73/200	0.096%	161/200

The comparison is descriptive and clustered. BLS is lower on 15 instance means, higher on one (`chr22a`), and equal on four. The paired mean difference is -0.205 percentage points. A cluster-by-instance bootstrap gives a 95% interval of -0.300 to -0.111. BLS is lower on 15 of 16 non-tied instance means; the family-cluster interval is not reported because it rests on only three families. The effective sample is 20 instances in three families, not 200 independent runs, so the instance-clustered interval is descriptive. Equal-weighting the families gives 0.099% for BLS and 0.298% for tabu, so the advantage is not only the eight `bur26` variants, which contribute 80 of the 200 runs.

The tabu reference is weak on the asymmetric `bur26` family: it reached no optimum in 80 runs, while BLS reached all 80. We report this as a property of this untuned reference implementation, not as evidence about the tabu reference in general. We therefore treat BLS as the primary QAP quality reference and the tabu path as an unvalidated reference implementation. The panel is a negative result for the tabu reference and a positive result for the harness: the harness made a stronger baseline visible, replayable, and comparable on the same denominator. Neither method is claimed to be state of the art.

3.3 TSP breadth panel

The TSP breadth panel uses 20 TSPLIB instances, five seeds each, a 10,000-kick cap, a 1,000-kick patience rule, and a 3,600-second safety cap. It is a frozen panel with per-instance maximum exchange depth: depth 4 for 17 instances, depth 3 for `lin105`, and depth 2 for `d198` and `pr152`. It is not one adaptive-depth experiment, and we do not interpret depth as a controlled factor. The panel combines runs produced under a legacy depth-4 setting and later runs at reduced depth after the original depth-4 pass on `d198` exceeded

300 seconds per kick; the recorded depth is a property of each row, and the quality comparison is across the panel as frozen.

Ninety-five of 100 runs stopped on patience, and 94 reached the known optimum. The mean gap is 0.004% and the worst 0.185%. Five `brazil58` runs reached the known optimum but hit the 3,600-second cap after roughly 380–400 kicks; they are reported as cap hits, not convergence. `brazil58` ($n = 58$, explicit upper-row matrix) is about four times slower per kick than the larger `bier127` under this implementation, so the cap is a real limit of the current path, not a solver failure hidden in a footnote. Table 2 keeps the cap hits in the denominator.

Table 2: Frozen TSPLIB breadth panel with per-instance exchange depth. Twenty instances, five seeds, a 10,000-kick cap, a 1,000-kick patience rule, and a 3,600-second safety cap. The five `brazil58` runs (depth 4) reached the optimum but hit the wall-clock cap and are not counted as patience stops. Gaps are percentages to the known optimum.

Depth	Instances	Runs	Mean gap	Worst gap	Patience	Optima
4	17	85	0.000%	0.000%	80/85	85/85
3	1	5	0.000%	0.000%	5/5	5/5
2	2	10	0.039%	0.185%	10/10	4/10
All	20	100	0.004%	0.185%	95/100	94/100

For context, a separate four-instance deep panel (`berlin52`, `kroA100`, `kroB100`, `ch150`) used 10,000 kicks and a 1,000-kick patience rule from converged local-search tours; all 20 runs stopped on patience, reached the optimum, and replayed. The breadth panel is not a replacement for that controlled comparison: it uses nearest-neighbour starts, and its mixed depth means the absolute gaps are descriptive.

3.4 Broad screens, external LKH, and a generic negative result

Two fixed-budget screens test coverage. The QAP screen parses 134 QAPLIB instances with five seeds and a 600-second cap; 552 of 670 runs stopped on patience and 318 reached the known optimum. The converged subset has median gap 0.000%, mean 1.823%, and worst 36.445%; the mean is pulled by a small hard tail, and the median is not evidence that the tail is solved. The TSP screen covers 78 instances with $n \leq 1000$; 236 of 390 runs stopped on patience and 195 reached the optimum, with an all-run mean of 1.460%. Above $n = 300$, only 3 of 100 runs stopped on patience, 1 reached the optimum, and 97 hit the safety cap. The internal method is therefore a small-instance solver with a weak larger tail (the evidence records retain the full distribution).

External LKH [25] is an optional capability reference, not part of the package. On the same 20-instance large tail, at a 600-second timeout, five seeds, and one run per seed (LKH 3.0.6, commit `9a51de3`), it reached 75 of 100 optima with a mean gap of 0.098%, median 0%, and worst 1.930%; mean wall time was 15.0 seconds. On that tail, FRESH reached 1 of 100 optima and hit the cap on 97 runs. The comparison is capability-only, not matched time; LKH is research-use software and is not bundled.

Finally, a generic tabu/large-neighbourhood prototype used only adapter proposals, apply/undo, and exact evaluation. It ran on four QAP and four TSP instances, five seeds, 20,000 iterations, 16 proposals per iteration, and a 5,000-iteration patience rule. All 40 runs stopped on patience and replayed exactly, and the result is negative: the QAP mean gap was 3.848% against 0.097% for the adapter-specific tabu reference, and the TSP mean gap was 127.978% against 0.000% for the LK reference. The generic runs start from random assignments and use about 117,000 exact evaluations, so this is not a matched-work or matched-start comparison. It supports a narrow statement only: this configuration is not competitive with a problem-specific neighbourhood, and the adapter contract is not by itself a search engine.

4 Availability and reuse

FRESH Solutions is developed by the UBC FRESH Lab. The code is at `https://github.com/UBC-FRESH/fresh-solutions`, release candidate `v0.9.3-p13rc1`. The paper-specific evidence is public at

https://github.com/UBC-FRESH/fresh-solutions-papers/releases/tag/p14-evidence-rc11. A DOI will be minted when the release is accepted; the GitHub release is a public prerelease.

FRESH Solutions is licensed under Apache-2.0. The paper-specific runners and derived evidence are intended for release under CC BY 4.0; code in the papers repository is MIT. The four QAP and four TSP instance files in paper2/instances are included under their original benchmark terms; QAPLIB and TSPLIB are not relicensed here. External LKH is research-use software and is not bundled; its provenance sidecar records the version, commit, binary hash, and rerun command.

The campaign environment used Python 3.12; the package requires Python 3.11 or later. It uses NumPy [26] and SciPy [27] for the independent QAP baseline. Numba [28] is optional for the accelerated QAP path; the reference path runs without it. No GPU is required. A Dockerfile and requirements file are included; the base image and solver checkout are pinned, but we have not executed a container build in this environment.

After mounting QAPLIB and TSPLIB, one command replays the evidence:

```
export QAPLIB_ROOT=/path/to/QAPLIB; export TSPLIB_ROOT=/path/to/tsplib;
bash paper2/replay_all.sh
```

The script re-evaluates 40 framework, 20 deep-TSP, 200 expanded-QAP, 200 BLS, 100 expanded-TSP, 40 generic, and 1,060 broad incumbents. Twenty-five esc8 parse-error records are skipped explicitly, and the five brazil58 cap hits are reported as cap hits. The validator paper2/check_evidence.py checks provenance, record counts, stop reasons, and replay status; the individual replay scripts perform the full adapter re-evaluation.

The repository and tag are public; a DOI and a tested container digest remain pending. Per-record CPU time is absent from the expanded-QAP and expanded-TSP records (the BLS and generic records carry it), so CPU timing is reported only where recorded.

Reuse is the point of the design. A new adapter for a compiled spatial model can reuse the evidence machinery, the replay gate, the stopping taxonomy, and the run/summary schema while supplying its own neighbourhood; the two reference adapters are templates, not a closed list. A team that already has a mature solver and a mature evidence pipeline should not adopt the boundary for its own sake. A team that rebuilds replay and provenance code for every problem is the intended user.

5 Limitations and claim boundary

The controlled framework panel has four QAP instances. The quality panel has 20 instances in three families, with a BLS baseline that is stronger than the tabu reference; the tabu result is a negative result, not a baseline endorsement. The QAP reference is untuned and weak on asymmetric instances, and BLS is a reimplementation from the published design, not the original authors' code.

The TSP breadth panel is frozen and mixed-depth, not a controlled depth comparison; its 3,600-second cap is close enough to normal runtimes that a slightly slower host could change the patience count. The deep panel has four instances. The internal LK exchange is bounded and lacks α-nearness, fractional cascading, and the full LKH gain criteria, which is why the larger tail is weak. The LKH comparison is capability-only and not matched time.

Timings come from a shared 72-logical-CPU Linux host under parallel load. The framework comparison alternates execution order by seed and uses process CPU time, but wall times are descriptive, not single-thread benchmarks. Production-path CPU time is not recorded, so the overhead is scoped to the private-array baseline.

Most importantly, portability is demonstrated by two adapters written by the reference author, not by a third party. Two adapters are not proof of generality. A third-party adapter, a matched-time comparison against LKH, and a family-balanced panel are future work. We make no state-of-the-art claim for the tabu reference or for the LK implementation, and no claim that the harness is a general algorithm-selection framework.

6 Conclusion

FRESH Solutions provides a benchmark harness and adapter boundary, with two reference metaheuristic implementations, exact replay, and a bounded framework-use result for engine-managed QAP state. The QAP panel shows that a patience rule changes the reported gaps relative to a short fixed budget, while the framework path preserves the trajectory at a measured 1.24 CPU ratio. The TSP mixed-depth breadth panel reaches 94 of 100 known optima and keeps five cap hits visible. The generic layer is reported as a negative result rather than promoted. The next step is not a larger claim: it is a third-party adapter, a pinned release with a DOI and tested container, and matched-time comparisons. The harness is useful now because it makes those steps replayable.

Author contributions

Gregory Paradis: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review and editing, Visualization, Project administration.

Funding

No specific grant from a funding agency supported this work.

Competing interests

The author declares no competing interests.

Data and code availability

The harness and reference implementations are in FRESH Solutions `https://github.com/UBC-FRESH/fresh-solutions`, release candidate `v0.9.3-p13rc1`. The runners, replay scripts, and evidence are in `https://github.com/UBC-FRESH/fresh-solutions-papers`, public release candidate `p14-evidence-rc11`; a DOI is pending. QAPLIB and TSPLIB are public. Full per-instance tables for the QAP and TSP panels are provided in the supplementary material.

References

[1] J. N. Hooker. Testing heuristics: We have it all wrong. *Journal of Heuristics*, 1(1):33–42, 1995. doi: 10.1007/BF02430364.

[2] Richard S. Barr, Bruce L. Golden, James P. Kelly, Mauricio G. C. Resende, and William R. Stewart. Designing and reporting computational experiments with heuristic methods. *INFORMS Journal on Computing*, 7(1):1–17, 1995.

[3] Catherine C. McGeoch. Toward an experimental method for algorithm simulation. *INFORMS Journal on Computing*, 8(1):1–15, 1996. doi: 10.1287/ijoc.8.1.1.

[4] Vahid Beiranvand, Warren Hare, and Yves Lucet. Best practices for comparing optimization algorithms. *Optimization and Engineering*, 18(4):815–848, 2017. doi: 10.1007/s11081-017-9366-1.

[5] Thomas Bartz-Beielstein, Carola Doerr, Daan van den Berg, Jakob Bossek, Sowmya Chandrasekaran, Tome Eftimov, Andreas Fischbach, Pascal Kerschke, William La Cava, Manuel Lopez-Ibanez, Katherine M. Malan, Jason H. Moore, Boris Naujoks, Patryk Orzechowski, Vanessa Volz, Markus Wagner, and Thomas Weise. Benchmarking in optimization: Best practice and open issues, 2020.

[6] Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. COCO: a platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 36(1):114–144, 2020. doi: 10.1080/10556788.2020.1808977.

[7] Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Mauro Birattari, and Thomas Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016. doi: 10.1016/j.orp.2016.09.002.

[8] Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.

[9] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.

[10] Greg Wilson, D. A. Aruliah, C. Titus Brown, Neil P. Chue Hong, Matt Davis, Richard T. Guy, Steven H. D. Haddock, Kathryn D. Huff, Ian M. Mitchell, Mark D. Plumbley, Ben Waugh, Ethan P. White, and Paul Wilson. Best practices for scientific computing. *PLoS Biology*, 12(1):e1001745, 2014. doi: 10.1371/journal.pbio.1001745.

[11] Victoria Stodden, Marcia McNutt, David H. Bailey, Ewa Deelman, Yolanda Gil, Brooks Hanson, Michael A. Heroux, John P. A. Ioannidis, and Michela Taufer. Enhancing reproducibility for computational methods. *Science*, 354(6317):1240–1241, 2016. doi: 10.1126/science.aah6168.

[12] David L. Donoho, Arian Maleki, Inam Ur Rahman, Morteza Shahram, and Victoria Stodden. Reproducible research in computational harmonic analysis. *Computing in Science & Engineering*, 11(1):8–18, 2009. doi: 10.1109/MCSE.2009.15.

[13] Carl Boettiger. An introduction to docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1):71–79, 2015. doi: 10.1145/2723872.2723882.

[14] Gregory M. Kurtzer, Vanessa Sochat, and Michael W. Bauer. Singularity: Scientific containers for mobility of compute. *PLoS ONE*, 12(5):e0177459, 2017. doi: 10.1371/journal.pone.0177459.

[15] Rainer E. Burkard, Stefan E. Karisch, and Franz Rendl. QAPLIB – a quadratic assignment problem library. *Journal of Global Optimization*, 10:391–403, 1997. doi: 10.1023/A:1008293323270.

[16] Gerhard Reinelt. TSPLIB – a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991. doi: 10.1287/ijoc.3.4.376.

[17] Christian Blum and Andrea Roli. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys*, 35(3):268–308, 2003. doi: 10.1145/937503.937505.

[18] Holger H. Hoos and Thomas Stützle. *Stochastic Local Search: Foundations and Applications*. Morgan Kaufmann, 2004.

[19] Éric D. Taillard. Robust taboo search for the quadratic assignment problem. *Parallel Computing*, 17(4–5): 443–455, 1991. doi: 10.1016/S0167-8191(05)80147-4.

[20] Shen Lin and Brian W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516, 1973. doi: 10.1287/opre.21.2.498.

[21] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220 (4598):671–680, 1983. doi: 10.1126/science.220.4598.671.

[22] El-Ghazali Talbi. *Metaheuristics: From Design to Implementation*. Wiley, 2009.

[23] David Pisinger and Stefan Ropke. Large neighborhood search. In Michel Gendreau and Jean-Yves Potvin, editors, *Handbook of Metaheuristics*, pages 399–419. Springer, 2010.

[24] Una Benlic and Jin-Kao Hao. Breakout local search for the quadratic assignment problem. *Applied Mathematics and Computation*, 219(9):4800–4815, 2013. doi: 10.1016/j.amc.2012.10.106.

[25] Keld Helsgaun. LKH-3: an extension of the lin–kernighan–helsgaun TSP solver. `https://github.com/cerebis/LKH3`, 2019. External research-use software; not bundled.

[26] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[27] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[28] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: a LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pages 1–6, 2015. doi: 10.1145/2833157.2833162.